

Compiling Typed APIs into Context-Free Languages

Breandan Considine, Clemente Pasti, Tim Vieira

August 10, 2026

Motivation: type-safe code completion

- ▶ Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- ▶ No well-typed statement $\sigma \notin \ell_\Gamma$ or ill-typed statements $\sigma \in \ell_\Gamma$
- ▶ For any $\sigma \in \Sigma^*$ such that $\exists \tau. \sigma\tau \in \ell_\Gamma$, show τ 's in shortlex order

Motivation: type-safe code completion

- ▶ Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- ▶ No well-typed statement $\sigma \notin \ell_\Gamma$ or ill-typed statements $\sigma \in \ell_\Gamma$
- ▶ For any $\sigma \in \Sigma^*$ such that $\exists \tau. \sigma\tau \in \ell_\Gamma$, show τ 's in shortlex order

```
1 #include <memory>
2 long* codes(); double* groups();
3 const char* text(); float* samples();
4 std::unique_ptr<int> fresh(), parse(const char*);
5 std::unique_ptr<int> average(float*), select(double&);
6 std::unique_ptr<int> combine(const char*, long*);
7 std::unique_ptr<int> schedule(const char*, long*, float*);
8
9 int main(int argc, char** args) {
10     std::unique_ptr<int> b |
11 }
```

Motivation: type-safe code completion

- ▶ Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- ▶ No well-typed statement $\sigma \notin \ell_\Gamma$ or ill-typed statements $\sigma \in \ell_\Gamma$
- ▶ For any $\sigma \in \Sigma^*$ such that $\exists \tau. \sigma\tau \in \ell_\Gamma$, show τ 's in shortlex order

```
1 #include <memory>
2 long* codes(); double* groups();
3 const char* text(); float* samples();
4 std::unique_ptr<int> fresh(), parse(const char*);
5 std::unique_ptr<int> average(float*), select(double&);
6 std::unique_ptr<int> combine(const char*, long*);
7 std::unique_ptr<int> schedule(const char*, long*, float*);
8
9 int main(int argc, char** args) {
10     std::unique_ptr<int> b |
11 }
```

```
= fresh();
= average(samples());
= parse(text());
= select(groups()[argc]);
= combine(text(), codes());
= schedule(text(), codes(), samples());
```

How long are typical source code statements?

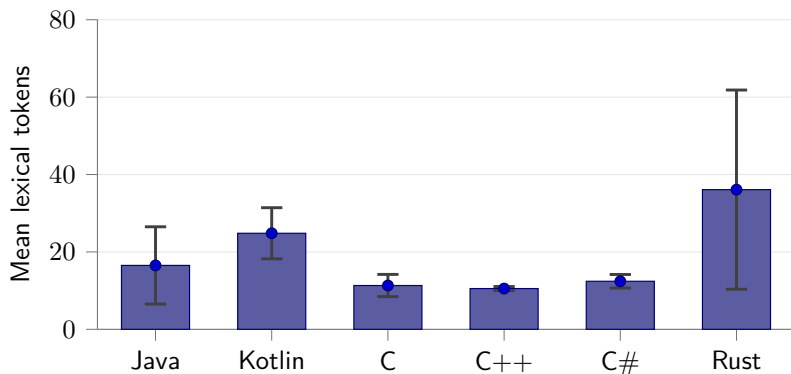


Figure: Whiskers are 95% CIs over sampled files. Statements are Tree-sitter statement nodes with comments and whitespace ignored. Sample size is 100 random GitHub files per language > 20kb.

Correctness criterion: what must the CFG preserve?

Surface fragment G'

e.g., names, invocation, member chains

$S ::= E; \quad E ::= \text{ID} \mid \text{ID}() \mid \text{ID}(\text{ARGS}) \mid E.E \quad \text{ARGS} ::= E \mid E, \text{ARGS}$

1. Surface projection π_Σ

erase names but preserve punctuation

$$\pi_\Sigma(\mathbf{x.f(y)}) = \text{ID.ID(ID)}$$

checks that the shape belongs to G_0

2. Typed projection Π_Γ

names $\rightarrow$ type slots; API names stay

$$\Pi_\Gamma(\mathbf{x.f(y)}) \ni E\langle\hat{X}\rangle.f(E\langle\hat{Y}\rangle)$$

contains all and only well-typed terms

Correctness criterion

For a target language L with surface fragment G' , construct G_Γ satisfying:

$$\pi_\Sigma(\sigma) \in \mathcal{L}(G') \iff \Gamma \vdash \Pi_\Gamma(\sigma) \in \mathcal{L}(G_\Gamma)$$

for all source code statements σ accepted by the reference compiler for L .

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

$G_{C++} \cup$

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

$G_{\text{C++}} \cup$

$G_{\text{<string>}} \cup$

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

$G_{\text{C++}} \cup$

$G_{\text{<string>}} \cup$

$G_{\text{<vector>}} \cup$

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"

using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;
// ...

int fizz() { return 3; }
string buzz(string s) { return s; }
// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
}
```

Grammar contribution

$G_{\text{C++}} \cup$
 $G_{\text{<string>}} \cup$
 $G_{\text{<vector>}} \cup$
 $G_{\text{org/fizz.hpp}} \cup$
 $G_{\text{com/buzz.hpp}} \cup$
 $\{ \text{Type} \rightarrow \text{string} \} \cup$

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
{ Type → string } ∪
...
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }
string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

```
 $G_{\text{C++}} \cup$   
 $G_{\text{<string>}} \cup$   
 $G_{\text{<vector>}} \cup$   
 $G_{\text{org/fizz.hpp}} \cup$   
 $G_{\text{com/buzz.hpp}} \cup$   
 $\{ \text{Type} \rightarrow \text{string} \} \cup$   
...  
 $\{ \text{Int} \rightarrow \text{fizz()}, \text{S} \rightarrow \text{fizz();} \} \cup$ 
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
{ Type → string } ∪
...
{ Int → fizz(), S → fizz(); } ∪
{ Str → buzz( Str ), S → buzz( Str ); } ∪
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
{ Type → string } ∪
...
{ Int → fizz(), S → fizz(); } ∪
{ Str → buzz( Str ), S → buzz( Str ); } ∪
...
```


Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
{ Type → string } ∪
...
{ Int → fizz(), S → fizz(); } ∪
{ Str → buzz( Str ), S → buzz( Str ); } ∪
...
{ Int → argc, CharPtr → args[ Int ] } ∪
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp

#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;

// ...

int fizz() { return 3; }

string buzz(string s) { return s; }

// ...

int main(int argc, char* args[]) {
    string bang = "...";
    |
}
}
```

Grammar contribution

```
GC++ ∪
G<string> ∪
G<vector> ∪
Gorg/fizz.hpp ∪
Gcom/buzz.hpp ∪
{ Type → string } ∪
...
{ Int → fizz(), S → fizz(); } ∪
{ Str → buzz( Str ), S → buzz( Str ); } ∪
...
{ Int → argc, CharPtr → args[ Int ] } ∪
{ Str → bang } → GΓ
```

Grammar Modularization

C++ context

```
// Filetype: *.cpp
#include <string>
#include <vector>
#include "org/fizz.hpp"
#include "com/buzz.hpp"
using std::string;
// ...
int fizz() { return 3; }
string buzz(string s) { return s; }
// ...
int main(int argc, char* args[]) {
    string bang = "...";
    |
}
```

Grammar contribution

```
 $G_{\text{C++}} \cup$ 
 $G_{\text{<string>}} \cup$ 
 $G_{\text{<vector>}} \cup$ 
 $G_{\text{org/fizz.hpp}} \cup$ 
 $G_{\text{com/buzz.hpp}} \cup$ 
 $\{ \text{Type} \rightarrow \text{string} \} \cup$ 
...
 $\{ \text{Int} \rightarrow \text{fizz()}, \text{S} \rightarrow \text{fizz()}; \} \cup$ 
 $\{ \text{Str} \rightarrow \text{buzz( Str )}, \text{S} \rightarrow \text{buzz( Str )}; \} \cup$ 
...
 $\{ \text{Int} \rightarrow \text{argc}, \text{CharPtr} \rightarrow \text{args[ Int ]} \} \cup$ 
 $\{ \text{Str} \rightarrow \text{bang} \} \rightarrow G_{\Gamma}$ 
 $\mathcal{L}(G_{\Gamma})$ 
```

Monomorphization I: ground rules

State $\rightarrow$ **nonterminal**; API text $\rightarrow$ terminal; validated AST node $\rightarrow$ production.

Monomorphic member call

$$\frac{\Delta(\widehat{C}:f)=(\widehat{A})\rightarrow\widehat{R} \quad \Delta;\Gamma\vdash e_0:\widehat{C} \quad \Delta;\Gamma\vdash e_1:\widehat{A}}{\Delta;\Gamma\vdash e_0.f(e_1):\widehat{R}}$$

becomes

$$E\langle\widehat{R}\rangle \rightarrow E\langle\widehat{C}\rangle.f(E\langle\widehat{A}\rangle)$$

Example `C c; A a; c.f(a);`

Resolved operator call

$$\frac{\Delta;\Gamma\vdash e_1:\widehat{T}_1 \quad \Delta;\Gamma\vdash e_2:\widehat{T}_2 \quad \delta_+^\Delta(\widehat{T}_1,\widehat{T}_2)=\widehat{R}}{\Delta;\Gamma\vdash e_1+e_2:\widehat{R}}$$

becomes

$$E\langle\widehat{R}\rangle \rightarrow E\langle\widehat{T}_1\rangle+E\langle\widehat{T}_2\rangle$$

Example `Vec u, v; u + v;`

$$\text{AddClosed}(T) \iff \text{base}\left(\delta_+^\Delta(\widehat{T},\widehat{T})\right) = T \xrightarrow{\text{erase state}} E\langle T \rangle \rightarrow E\langle T \rangle + E\langle T \rangle$$

Type-level mnemonic only; G_0^+ preserves C++ precedence/left associativity, and mixed states use δ_+^Δ .

Monomorphization II: subtype calls

$D <:_\Delta B$ is public, unambiguous inheritance; whole-call resolution retains cv/ref/category state and conversions.

Resolve the subtype call

$$\frac{D <:_\Delta B \quad \Delta; \Gamma \vdash e : \hat{D} \quad \text{resolve}_\Delta(f, \hat{D}) = ((\hat{B}) \rightarrow \hat{R})}{\Delta; \Gamma \vdash f(e) : \hat{R}} \text{Inv}_{<:}$$

No global $e : B$ rule: invocation consumes the subtype fact.

Example

```
struct Animal {};  
struct Dog : Animal {};  
int age(const Animal&);  
Dog d; age(d);
```

Emit the exact surface call

$$\frac{\text{resolve}_\Delta(f, \hat{D}) = ((\hat{B}) \rightarrow \hat{R})}{(E\langle \hat{R} \rangle \rightarrow \mathbf{f} \ (E\langle \hat{D} \rangle)) \in P_{\text{api}, b}} \text{Inv}_\varphi$$

State hats are elided below.

$E\langle \text{int} \rangle \rightarrow \text{age} \ (E\langle \text{Dog} \rangle)$

$E\langle \text{int} \rangle \Rightarrow \text{age} \ (E\langle \text{Dog} \rangle),$
 $\text{age} \ (E\langle \text{Dog} \rangle) \in \Pi_\Gamma(\text{age}(d)).$

Whole-call resolution preserves overloads, access, cv/ref/category, and ambiguity.

`Dog <: Animal` does not lift through `vector<·>` (class-template invariance).

Monomorphizing templates

Proposed C++ grounding

$$\begin{aligned} & f: \forall \bar{\alpha}. C(\bar{\alpha}) \Rightarrow (\bar{\tau}) \rightarrow \rho \in \Delta \\ & \text{types}(f_{\theta}, \bar{q}, q_R) \subseteq \widehat{\mathcal{T}}_b \quad C\theta \\ & \text{oracle}_{\text{C++}}(f(\bar{e}:\bar{q})) = q_R \end{aligned}$$

$$f_{\theta, \bar{q}} : (\bar{q}) \rightarrow q_R \in \Delta_b \quad \text{Mono}$$

emit $E\langle q_R \rangle \rightarrow f(E\langle q_1 \rangle, \dots, E\langle q_n \rangle)$.

$q = (T, \text{cv/ref}, \kappa)$ is a complete expression state.

Tiny C++ schema

```
template<class T>
requires AddClosed<T>
T twice(T x) {
  return x + x;
}

twice :  $\forall T.$ 
      AddClosed( $T$ )  $\Rightarrow T \rightarrow T$ 
```

Validate exact surface calls; keep accepted rows

Substitution	C++ oracle	Production (states elided)
$T \mapsto \text{int}$	accepted	$E(\text{int}) \rightarrow \text{twice}(E(\text{int}))$
$T \mapsto \text{string}$	accepted	$E(\text{string}) \rightarrow \text{twice}(E(\text{string}))$
$T \mapsto \text{vector<int>}$	rejected	omitted: no closed operator+

$b = \langle \text{arity}, \text{nesting}, \text{subtype depth}, |\widehat{\mathcal{T}}_b| \rangle$. exact-call oracle $\Rightarrow$ sound for this schema.

Complete metadata + exhaustive in-bound calls $\Rightarrow$ complete for this schema in $(G_0^+, \widehat{\mathcal{T}}_b, b)$.

Covariance and contravariance

Results: Size of generated CFGs

Evaluation